Feature Selection

2026-04-17

Introduction

Feature selection retains the predictors most relevant to the target while discarding the rest. The benefits are simpler models, reduced over-fitting, faster inference, improved interpretability, and reduced annotation cost in some applications. Three families of methods coexist: filter (univariate tests, cheap and fast), wrapper (fit-and-evaluate, slow but thorough), and embedded (selection happens during fitting, balancing speed and quality). Modern ML pipelines almost always use embedded methods (lasso, tree importance, Boruta) because they integrate naturally with cross-validation.

Prerequisites

A working understanding of regression and classification basics, regularisation (L1/lasso), and the bias-variance trade-off.

Theory

Filter methods rank features by univariate statistics — Pearson or Spearman correlation, mutual information, chi-squared, ANOVA F-statistic — and select the top k. Fast but ignore feature interactions and joint information.

Wrapper methods treat the model as a black box, fitting and evaluating across feature subsets. Forward selection adds features one at a time; backward elimination starts with all features and removes them; recursive feature elimination (RFE) cycles through both with cross-validation. Slow but capable of finding complementary feature combinations.

Embedded methods integrate selection with fitting. Lasso drives some coefficients to exactly zero (selection); tree-based methods (random forest, XGBoost) provide importance scores; Boruta compares each feature against random permuted shadows in many resamples to test for genuinely predictive features. Stability selection runs lasso on many subsamples and ranks features by selection frequency.

Assumptions

Selected features generalise from training to deployment data; selection is done within CV folds to prevent leakage of test-fold information into the chosen feature set.

R Implementation

```r
library(Boruta); library(glmnet)

set.seed(2026)
n <- 300; p <- 20
X <- matrix(rnorm(n * p), n, p)
colnames(X) <- paste0("x", 1:p)
y <- X %*% c(rep(2, 5), rep(0, 15)) + rnorm(n)

# Embedded: Lasso selects non-zero features
```

```r
lasso <- cv.glmnet(X, y, alpha = 1)
nonzero_lasso <- rownames(coef(lasso, s = "lambda.min"))[
  which(as.vector(coef(lasso, s = "lambda.min")) != 0)
][-1]
cat("Lasso selected:", length(nonzero_lasso), "\n")

# Wrapper-ish: Boruta compares features to shadow permutations
bor <- Boruta(X, y, maxRuns = 50, doTrace = 0)
confirmed <- getSelectedAttributes(bor, withTentative = FALSE)
cat("Boruta confirmed:", length(confirmed), "\n")
```

Output & Results

Lasso retains the features whose coefficients survive the L1 penalty at the cross-validation-optimal λ. Boruta confirms or rejects features based on whether their importance exceeds that of randomly permuted shadow features across many resampling runs. On the sparse-signal dataset (5 true features, 15 noise), both methods typically recover 4–5 true features with near-zero false positives.

Interpretation

A reporting sentence: "Lasso (cross-validated $\lambda = 0.08$) and Boruta (50 runs, BH-adjusted importance against shadow features) independently identified the same 5 predictive features matching the data-generating model, with no false positives in either; correlation-based filter selection (top 5 by absolute correlation with y) missed one feature because its marginal correlation was masked by noise from collinear predictors." Always describe the selection method and any cross-validation embedding.

Practical Tips

- Perform feature selection *inside* cross-validation folds; selection on the full dataset before CV leaks information from test folds into the chosen feature set.
- Lasso is fast and principled for linear or near-linear relationships; random forest importance handles non-linear effects better.
- Boruta's multiple-run shadow-feature comparison is statistically cleaner than single-pass importance and provides confidence statements about selection.
- For $p > n$ regimes, lasso (or elastic net) is nearly mandatory; random forest importance alone becomes unreliable in very high-dimensional settings.
- Stability selection (`stabs::stabsel`) fits many lassos on data subsamples and ranks features by their selection frequency; this is a robust alternative when a single CV run is unstable.
- Pre-screen with cheap filter methods (variance, correlation) when p is in the millions; this reduces the candidate set before applying the more expensive wrapper or embedded methods.

R Packages Used

`glmnet` for lasso and elastic-net embedded selection; `Boruta` for shadow-feature-based confirmation; `stabs` for stability selection; `caret::rfe()` for recursive feature elimination; `vip` for permutation-based variable importance from any model; `recipes::step_select_*` for tidymodels-flavoured selection within preprocessing pipelines.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**

- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis